

Transient Object Removal from LiDAR Point Clouds using Machine Learning

Student name: Divya Natekar
Faculty Mentor: Prof. Debra Laefer (Ph.D.), NYU CUSP
Tools & Technologies : Python (Open3D, Matplotlib, NumPy), Machine Learning (Supervised KNN, Feature Engineering) and Cloudcompare

Introduction

LiDAR is a kind of technology that captures the 3D scans of buildings and its surrounding areas, through laser pulses. Although it is extremely effective in documenting urban landscapes, the number of repeated scans capture not only the infrastructure, but also the **transient elements** (such as pedestrians and vehicles) which are not permanently present in real time. These elements represent the **unwanted noise**, which increase the size of the dataset and introduce distortions in its analysis. To address this recurring problem in numerous point-cloud data scans, the solution was to develop a programming approach which will aid this noise removal process, by simulating the work of a **Terrestrial Laser Scanner (TLS)**. The goal is to replicate its scanning procedure while simultaneously removing the unwanted noise from the dataset.

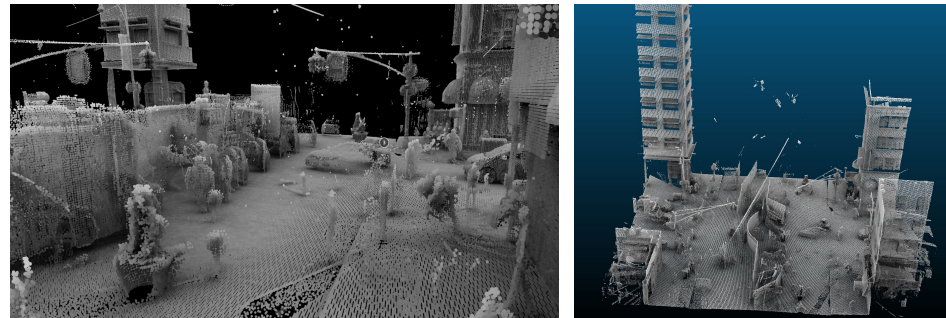


Fig.1. Original LiDAR Point Cloud (*jay-willoughby.ply*), as viewed in Cloudcompare.

Machine Learning

There are two ways to remove the noise. The first method involves manually filtering the noise, but can be extremely tedious and time-consuming. The second option is a much faster approach, by automating the filtering process through code. We can teach the program how to filter the points by providing it some references as inputs (labeled). This process is called supervised learning, and is a type of Machine Learning approach. The core principle of **Machine Learning** lies in a system's ability to learn how to predict the outcome, when it is trained via feeding it a series of labeled or unlabeled data, in the same manner as teaching a human being to perform a task. In our program, we can provide two references. The first will be an ideally cleaned dataset (*manual-filtering.ply*), while the second will be of only the noise points (*jw-noise-points.ply*).

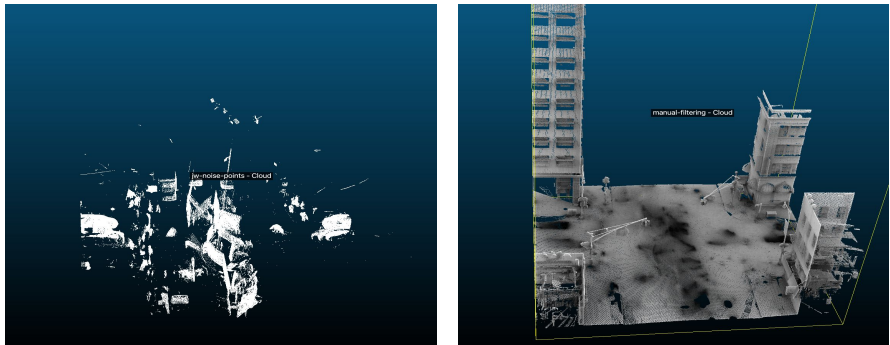


Fig.2. Reference datasets used for training — noise-only point cloud (left) and manually cleaned point cloud (right).

References

- Laefer, D. F. et al. (2025). *Low-Cost, LiDAR-Based, Dynamic Flood Risk Communication Viewer. Remote Sensing*, 17(4), 592.
- Nurunnabi, A. et al. (2024). *Development of a Precise Tree Structure from LiDAR Point Clouds. ISPRS Archives, XLVIII-2-2024*, 301–308.

Voxelization

The raw dataset (comprising 1.36M points) was downsampled to 1.29M points, by converting the points into a series of small 3D cubes (**voxels**). Machine Learning algorithms are known for their long execution time, and this approach helps in not only maintaining the integrity of the original dataset, but also enhancing the efficiency of the program.

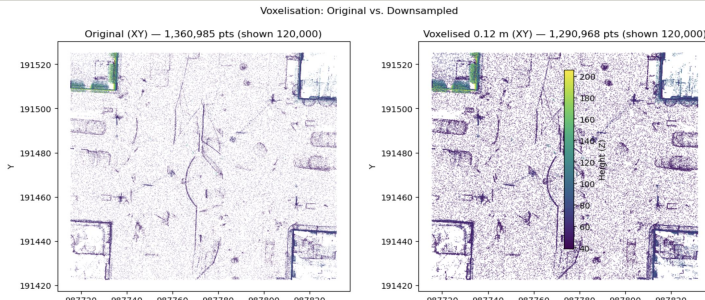


Fig.3. The comparison between the original (1.36 M points) and the voxelized version (1.29 M points).

Feature Engineering

The **geometric features** were extracted in order to aid the classification process. These features will enable the model to distinguish between the transient objects and the permanent structures:

- Height:** Used to distinguish between the low-lying road surfaces and the higher objects (i.e. pedestrians).
- Local density:** Helps in identifying the compact clusters (such as moving cars and people) from the sparsely distributed structural points.
- Normal Variation:** Captures how the surface orientation changes locally. The flat and continuous surfaces such as building facades show low variation, while the irregular scattered objects (such as people, vehicles and foliage) exhibit high variation.

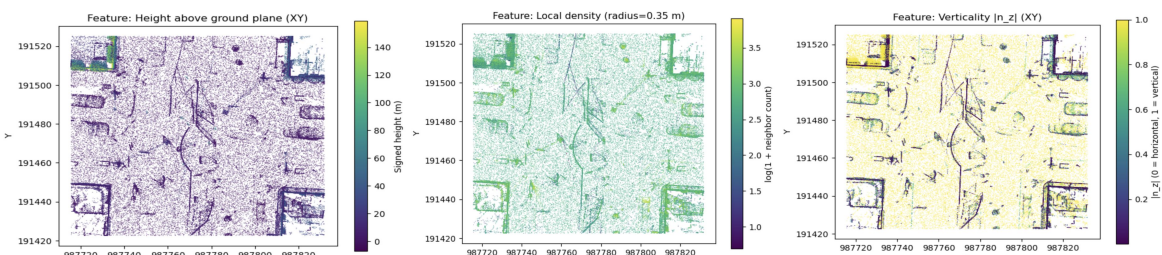


Fig.5. Multiple geometric features are derived from the point cloud to aid classification.

RANSAC

The **Random Sample Consensus (RANSAC)** algorithm was used to estimate and extract the ground plan. By iteratively fitting a plane model and rejecting outliers, RANSAC anchors the dataset at a street-level. This distinction between the ground inliers and the non-ground points, provides a baseline for separating the permanent infrastructure from the noise, above the ground plane.

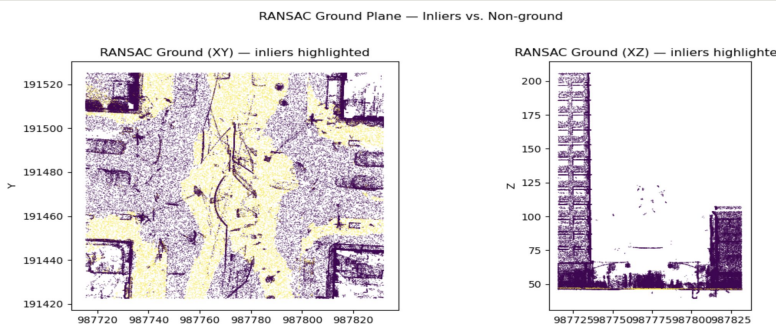


Fig.4. RANSAC segmentation separates the ground-plane inliers from non-ground points.

K-Nearest Neighbors (KNN)

The classifier selected for executing the program is the **K-Nearest Neighbors (KNN)** algorithm, for determining the class of a particular point based on the class of its nearest neighbors. The KNN algorithm is form of **supervised learning**, which means that it accepts labeled inputs for the purpose of training the model.

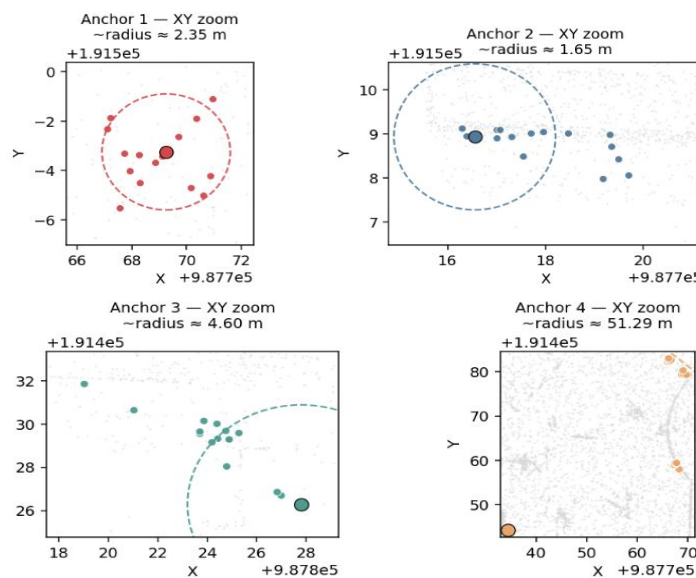


Fig.6. KNN classification illustrated for four anchor points with varying neighborhood radii.

Filtering of Transient Noise

For testing the program logic, a simple Machine Learning based model was developed, for replicating the working of a TLS scanner placed at Corner 1, and then moving clockwise through the subsequent corners (Corners 1→2→3→4), thus completing a loop. Four such loops (**16 iterations or stages**) were carried out, to progressively clean the dataset. The end result (*loop 4, corner 4*) strongly resembled the output of *manual-filtering.ply*, hence confirming the success of the program. The program was further extended to consider all the possible scanning orders. Within the four corner sequences, lie 24 possible permutations (i.e. 1→2→3→4, 1→2→4→3, 1→3→2→4, and so on), that are implemented as **24 Trials**. In order to validate the consistency, each sequence was repeated twice (Passes 'a' and 'b'), thus generating a total of 192 outputs (24 Trials x 2 Passes x 4 Corners = 192 Outputs).

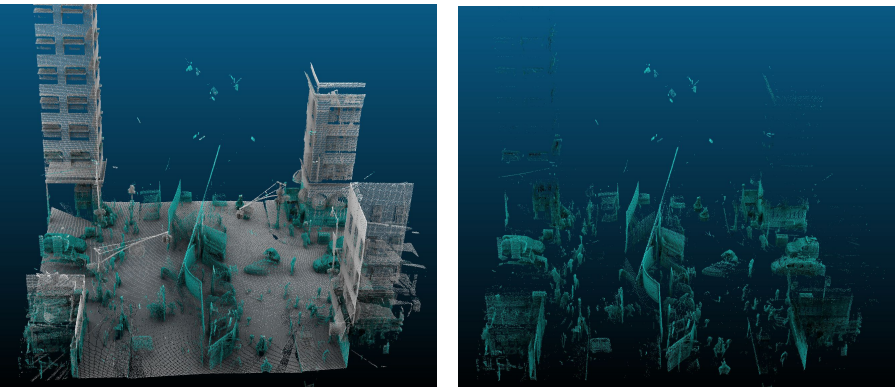


Fig.7. The results at the end of *loop 1, corner 1*: Retained structures (left) and Removed points (right).

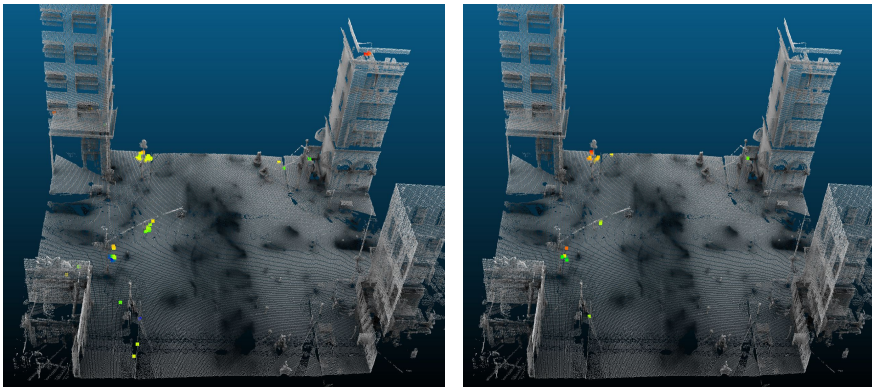


Fig.8. The results at the end of the *fourth iteration* under Trial 24, for both T1 (left) and T2 (right).

Results

Majority of transient objects were removed in the earlier passes (for e.g. in loop 1, corner 1 ~267,029 points (~20.2%) were eliminated in the first step, as compared to fewer than 300 points being eliminated in the later steps). This gives rise to an exponential decay graph (Fig. 9).

The number of points retained started to plateau after 10 iterations (Fig. 9) indicating that most of the noise had been filtered.

The point removal graph for Trials 1 and 2 (Fig. 10) showed a similar convergence behaviour, thus confirming the consistency across both Trials.

The stacked column (Fig. 10) demonstrates that the removals were evenly distributed across all 24 corner sequences.

As shown in the table (Fig. 11), the largest number of removals always occur in the first corner under the Pass 'a' (e.g. ~20,26% in Trial 2a corner 1), with the following steps contributing less than 2% each. This observation highlights the efficiency of the early passes in clearing the transient noise for the point-cloud dataset.

The exponential removal curve fits the data with a regression coefficient $R^2 \approx 1.000$, thus confirming that the observed convergence is mathematically precise and reproducible,

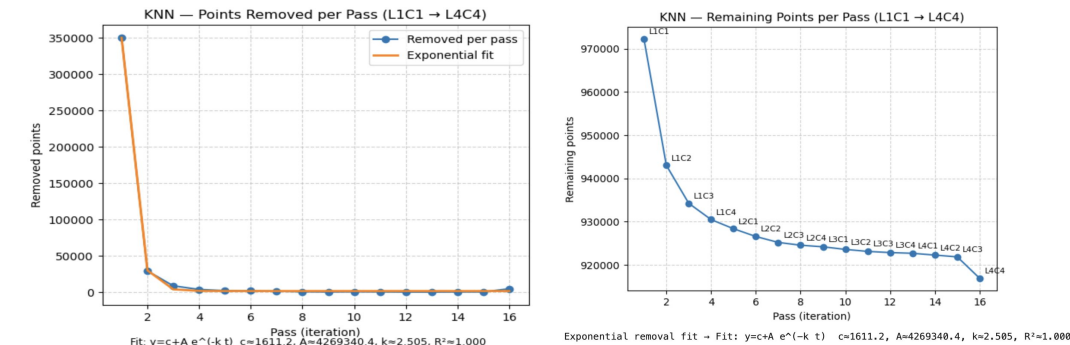


Fig.9. KNN filtering results across 16 passes, showing points removed (left) and points retained (right) with exponential fit.

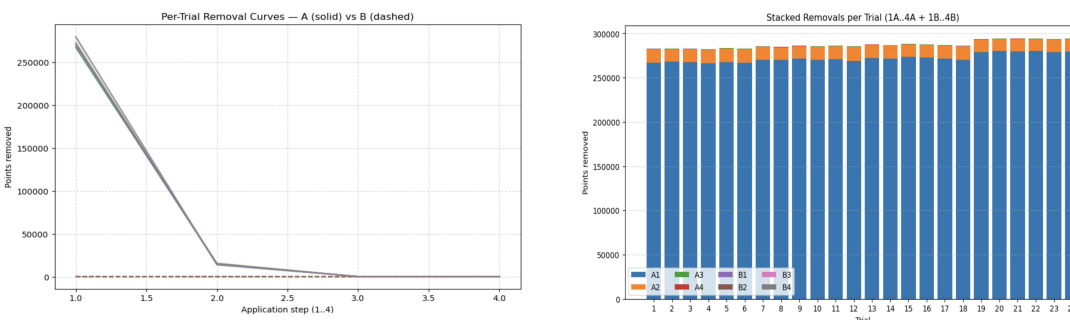


Fig.10. Extended KNN filtering across 24 trials with two passes, illustrating per-trial removals, and stacked removals.

trial	phase	step	corner	kept_after	removed_step	removed_pct_of_remaining	
0	1	a	1	1	1055517	267029	20.190526
1	1	a	2	2	1040397	15120	1.432473
2	1	a	3	3	1040102	295	0.028355
3	1	a	4	4	1039989	113	0.010864
4	1	b	1	1	1039950	39	0.003750
5	1	b	2	2	1039893	57	0.005481
6	1	b	3	3	1039866	27	0.002596
7	1	b	4	1	1039837	29	0.002789
8	2	a	1	1	1054591	267955	20.260543
9	2	a	2	2	1039985	14606	1.384992
10	2	a	3	4	1039688	297	0.028558
11	2	a	4	3	1039549	139	0.013369
12	2	b	1	1	1039489	60	0.005772
13	2	b	2	2	1039442	47	0.004521
14	2	b	3	4	1039423	19	0.001828
15	2	b	4	1	1039372	51	0.004907

$$P(4,3) = \frac{4!}{(4-3)!} = 24$$

Fig. 11. A sample of the program output stating the number of points removed for Trials 1 and 2, across applications a and b.

Contact

Divya Natekar
MS in Applied Urban Science & Informatics
Email: dyn2009@nyu.edu

